

# Introduction to Neural Networks

Andrew Ng

## Neural Networks – An Evolution of Machine Learning

Machine learning started with simple models such as **linear regression** and **logistic regression**, where we manually designed features and asked the model to learn the best weights. These models work well for simple problems, but they struggle when the data becomes **large, complex, or unstructured**, such as images, speech, and text.

As real-world problems became more complex, traditional machine learning models reached their limits. Tasks like recognizing faces, understanding spoken language, or translating text are very difficult to solve using hand-crafted rules or shallow models. We needed models that could **automatically learn complex patterns** directly from data.

Neural networks were introduced as the next step in this evolution. Instead of using one simple equation, neural networks **stack many layers of simple models together**. Each layer learns a different level of representation, starting from simple patterns and gradually building more meaningful features.

In this chapter, we will see how neural networks extend what we already know from logistic regression. We will learn how adding layers, nonlinear activation functions, and more parameters allows models to become **more powerful, more flexible, and better suited for modern AI applications**.

# Neural networks

Origins: Algorithms that try to mimic the brain.



Used in the 1980's and early 1990's.  
Fell out of favor in the late 1990's.

Resurgence from around 2005.

speech → images → text (NLP) → ...

## Neural Networks: Origins and Historical Development

Neural networks started as an idea inspired by the **human brain**.

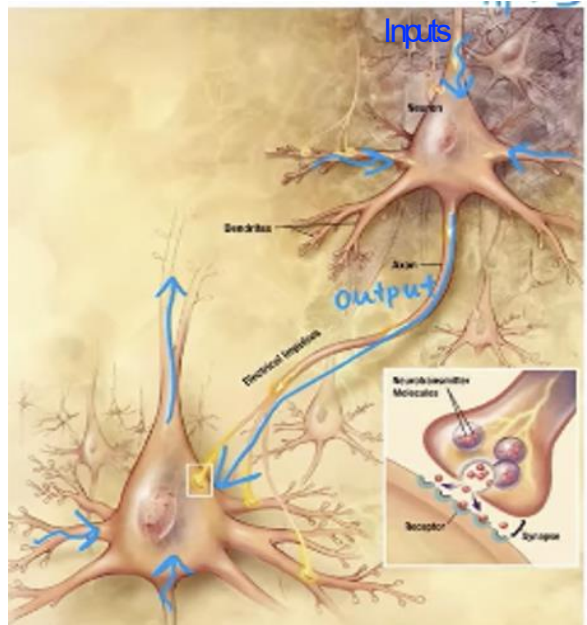
Researchers asked a simple question: *Can we design algorithms that learn in a way similar to how neurons in the brain work?* This idea led to early models that tried to imitate how biological neurons receive signals, process them, and produce outputs.

Neural networks were first actively used in the **1980s and early 1990s**. At that time, they showed promise, but they faced serious limitations.

Computers were slow, datasets were small, and training deep models was very difficult. Because of these challenges, neural networks **fell out of favor in the late 1990s**, and many researchers returned to simpler machine learning models.

Around **2005**, neural networks made a strong comeback. This resurgence happened because three key things changed: we had much **more data**, much **faster computers (GPUs)**, and **better training algorithms**. These advances made it possible to train larger and deeper networks effectively. Since then, neural networks have achieved major breakthroughs in **speech recognition**, then **image recognition**, and later **text and natural language processing (NLP)**. Today, they are the core technology behind modern AI systems, and their applications continue to expand into many other fields.

## Biological Neuron: How the Brain Processes Information



### Biological Neuron: How the Brain Processes Information

This slide shows a **biological neuron**, which is the basic building block of the human brain. A neuron's job is to **receive information, process it**, and then **send an output signal** to other neurons. This simple idea is the inspiration behind artificial neural networks.

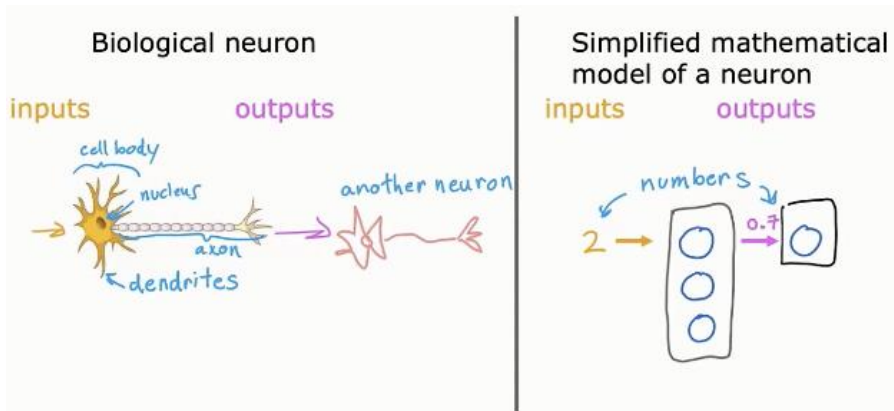
The **inputs** to the neuron come through the **dendrites**. These are like small branches that receive signals from other neurons. Each signal can be strong or weak, and many signals arrive at the neuron at the same time. The neuron does not react to just one input; it considers **all inputs together**.

Inside the neuron, these input signals are combined in the **cell body (soma)**. Here, the neuron decides whether the total signal is strong enough to take action. If the combined input passes a certain threshold, the neuron becomes active. If it does not, the neuron stays quiet. This idea of combining inputs and making a decision is **very important** in machine learning.

If the neuron activates, it sends an **output signal** through the **axon**. This output travels to other neurons and affects them. In this way, information flows through the brain as a network of connected neurons, where each neuron performs a small, simple computation.

Neural networks in machine learning are inspired by this process. Just like biological neurons, artificial neurons take inputs, combine them, apply a decision rule, and produce an output. By connecting many such neurons together, we can build powerful models that learn complex patterns from data.

# Biological vs. Artificial Neurons



Like biological neurons, artificial neurons work to process input data and Generates output, which is fed to the next neurons

Andrew Ng

## Biological Neurons vs. Artificial Neurons

This slide compares how a **real neuron in the brain** works with how we model a neuron **mathematically** in neural networks. The goal is not to copy the brain exactly, but to capture the **main idea of information processing** in a simple and useful way.

A **biological neuron** receives inputs through its **dendrites**. These inputs are signals coming from other neurons. Inside the **cell body**, the neuron combines all incoming signals. If the combined signal is strong enough, the neuron becomes active and sends an **output signal** through the **axon** to another neuron. This process allows information to flow through the brain.

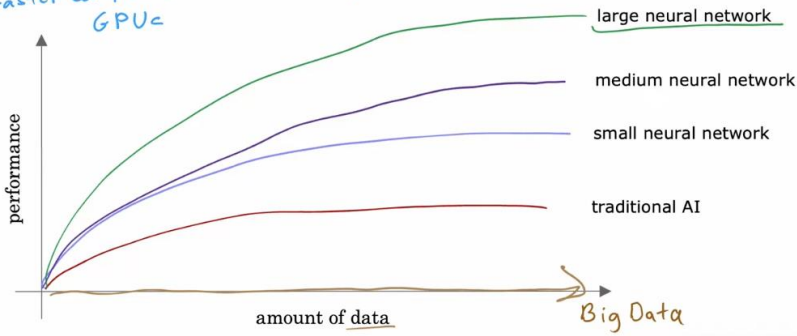
An **artificial neuron** follows the same high-level idea, but in a simplified mathematical form. Instead of biological signals, the inputs are **numbers** (for example: pixel values, prices, or sensor readings). Each input represents information coming into the neuron.

Inside the artificial neuron, these input numbers are combined using a mathematical rule. The neuron then produces a **single numerical output**, often a value between 0 and 1. This output is passed to the next neuron, just like signals move from one biological neuron to another.

The key takeaway is that **both types of neurons do the same job conceptually**: they take multiple inputs, process them together, and produce an output. By connecting many artificial neurons together, we create neural networks that can learn complex patterns from data.

## Why Now?

Faster computer processors  
GPU



Neural networks are the mechanism that is able to cope with the increasing Amount of data and the faster computing powers

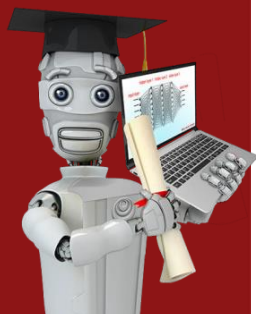
Andrew Ng

### Why Now? Why Neural Networks Work Well Today

This slide explains why neural networks are successful **now**, even though the idea is old.

The amount of data has increased a lot in recent years (**Big Data**). At the same time, computers have become much faster, especially with **GPUs**. Traditional AI models and small machine learning models improve only a little when more data is added. Their performance reaches a limit quickly. Neural networks, especially **larger and deeper networks**, continue to improve as we give them more data and more computing power.

**Key idea:** Neural networks can take advantage of **big data** and **fast computers**, which is why they perform so well today.



# Neural Network Intuition

## Demand Prediction

Andrew Ng

### Neural Network Intuition: Demand Prediction

This section introduces **neural networks through intuition**, using a simple and practical example: **demand prediction**.

The goal is to understand **how a neural network thinks**, not to focus on formulas yet. We want to see how a network can take several inputs, combine them, and make a useful prediction.

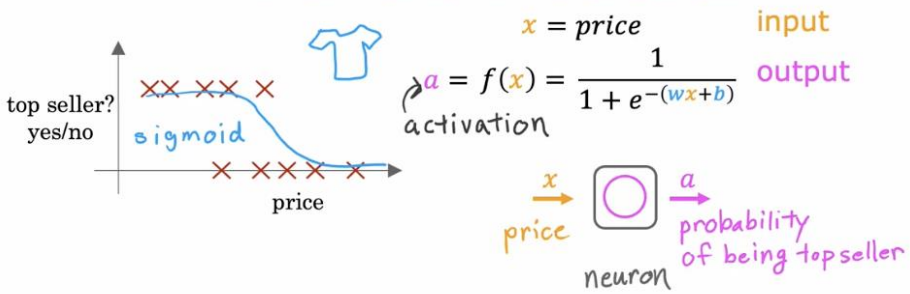
Demand prediction is a good example because it is familiar and easy to understand. A neural network can look at factors such as price, marketing, or other features and decide whether the demand for a product will be **high or low**.

In the next slides, we will see how a neural network:

- takes real-world inputs
- processes them step by step
- outputs a clear prediction

This example will help build intuition before moving to more complex cases like images, speech, and text.

# Demand Prediction



The sigmoid function which is used to predict whether a T-shirt is top Seller or Not (classification) is the work of a single Neurin on Neural Networks

## Demand Prediction: Neural Network Intuition (Single Neuron)

This slide shows how a **single neuron** can be used to solve a **demand prediction** problem. The goal is to predict whether a T-shirt will be a **top seller (yes or no)** based on its **price**.

The **input  $x$**  is the price of the T-shirt. This input is given to one neuron. Inside the neuron, the price is multiplied by a weight, a bias is added, and the result is passed through the **sigmoid function**.

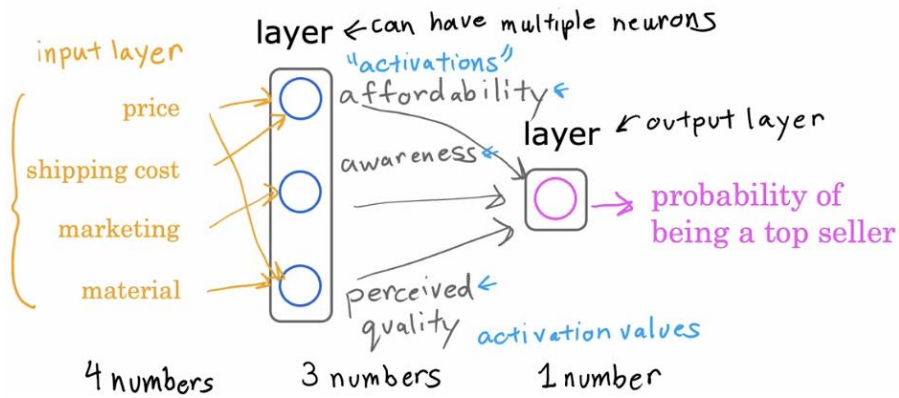
The sigmoid function produces an **output  $a$**  between 0 and 1. This output is interpreted as the **probability of being a top seller**. For example, a value close to 1 means high demand, while a value close to 0 means low demand.

Because the output is a probability, we can apply a threshold (such as 0.5) to make a final decision:

- $a \geq 0.5 \rightarrow$  top seller (yes)
- $a < 0.5 \rightarrow$  not a top seller (no)

The key idea of this slide is that a **single neuron with a sigmoid activation is enough to perform binary classification**. This is exactly the same idea as **logistic regression**, and it is the basic building block of neural networks.

# Demand Prediction



Here each neuron value like affordability is decided by the input features, in this case Price and shipping cost. The outcome of this neuron is called Activation

## Demand Prediction: Neural Network Intuition (Multiple Neurons)

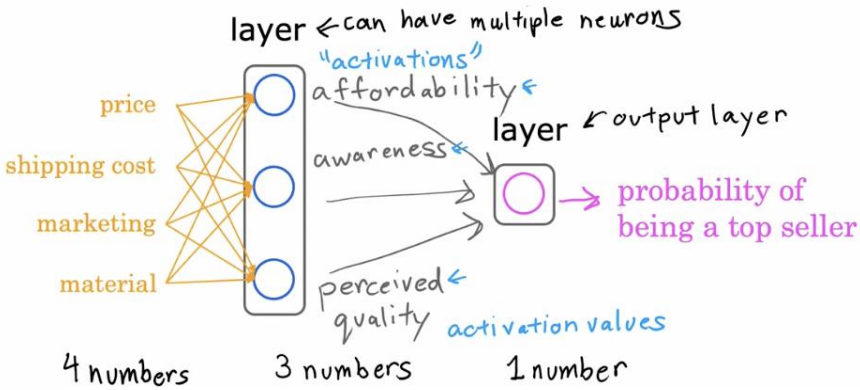
This slide shows how a neural network can make a **demand prediction** using **multiple inputs and multiple neurons**, instead of just one neuron. The **input layer** contains several input features. In this example, the inputs are **price**, **shipping cost**, **marketing**, and **material**. These are four numbers that describe the product. Each input feature provides different information that may affect demand.

The next layer is a **hidden layer**, and it can have **multiple neurons**. Each neuron in this layer looks at the same input features, but each one focuses on a **different idea**. For example, one neuron may learn **affordability**, another may learn **awareness**, and another may learn **perceived quality**. Each neuron combines the input features using weights and a bias, then produces an **activation value**. This activation value is a number that represents how strong that concept is. These activation values are the **outputs of the hidden layer**.

The **output layer** takes these activation values as input and produces **one final number**. This number represents the **probability that the product will be a top seller**.

The key idea is that **hidden layer neurons automatically learn useful concepts** from the input features. Instead of manually defining affordability or quality, the neural network **learns them by itself** through activations. This is what makes neural networks powerful for real-world prediction problems.

# Demand Prediction



Instead of deciding the features that affect each node in the hidden layer, we input All features to each node and let the nodes decide which features affect them more

## Demand Prediction: Let the Network Learn What Matters

This slide explains an important idea in neural networks: **we do not manually decide which input features affect each neuron**. Instead, we give **all input features to every neuron**, and let the network learn what is important by itself.

The **input layer** contains four features: **price**, **shipping cost**, **marketing**, and **material**. These are four numbers that describe the product. All of these inputs are connected to **every neuron** in the hidden layer.

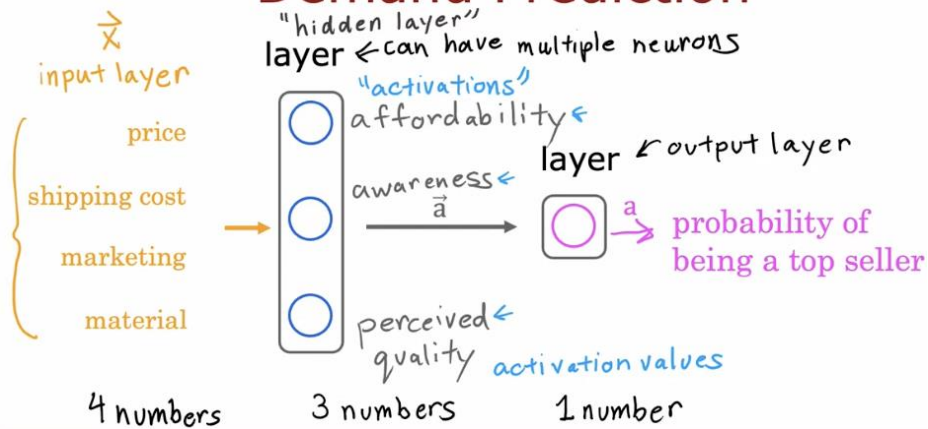
The **hidden layer** has multiple neurons. Each neuron receives **all input features**, but each neuron learns **different weights**. Because of this, one neuron may become sensitive to **affordability**, another to **awareness**, and another to **perceived quality**. These concepts are **not programmed by us**; they are learned automatically during training.

Each hidden neuron produces an **activation value**, which is a number representing how strong that learned concept is. Together, these activation values form the output of the hidden layer.

The **output layer** takes these activation values and produces **one final number**. This number represents the **probability that the product will be a top seller**.

The key idea of this slide is that neural networks **learn feature importance automatically**. We do not tell the model which inputs matter more. Instead, we provide all inputs, and the network decides—through learning—which features influence each neuron the most.

# Demand Prediction



The values of the input layer, the hidden Layer are treated as vectors, while the output is a single value

## Demand Prediction: Inputs, Hidden Layer, and Output as Vectors

This slide highlights an important way to think about neural networks: **inputs and hidden layers are vectors, while the output is a single value.**

The **input layer** contains several features: **price, shipping cost, marketing, and material**. These four values are grouped together and treated as **one input vector  $x$** . The network does not see them as separate items, but as a single vector of numbers.

The **hidden layer** also produces a **vector of values**. Each neuron in the hidden layer computes an activation value, such as affordability, awareness, or perceived quality. All these activation values together form the **activation vector  $a$**  of the hidden layer.

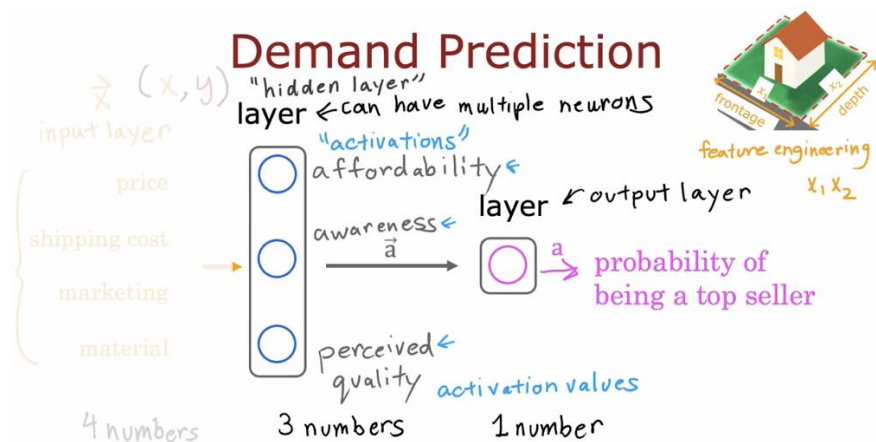
The **output layer** is different. It produces **one single number**, not a vector. This value represents the **probability that the product will be a top seller**.

The key idea is:

- Input layer → **vector of numbers**
- Hidden layer → **vector of activation values**
- Output layer → **single scalar value**

This vector-based view makes neural networks easier to understand and prepares students for writing neural networks using mathematical notation and code.

# Demand Prediction



The hidden layer works as one type of Feature Engineering

## Demand Prediction: Hidden Layer as Automatic Feature Engineering

This slide explains a key intuition about neural networks: the **hidden layer acts as a form of feature engineering**.

In traditional machine learning, we often create new features by hand. For example, from raw inputs we may design features such as affordability or quality. This process is called **feature engineering**, and it requires human effort and domain knowledge.

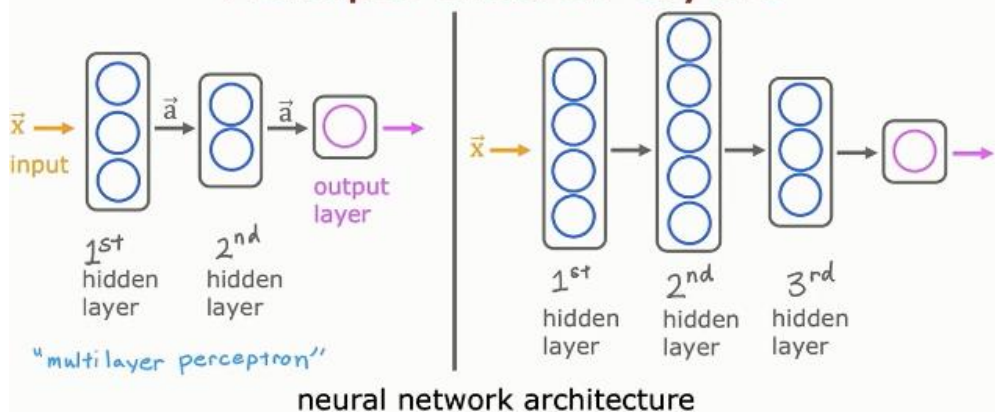
In a neural network, the **hidden layer performs this automatically**. The input features (price, shipping cost, marketing, material) are given directly to the network. Each neuron in the hidden layer combines these inputs and produces an **activation value**. These activation values represent **new, learned features** such as affordability, awareness, or perceived quality. These learned features are not explicitly defined by us. The network discovers them during training by adjusting weights and biases. The set of activation values produced by the hidden layer becomes a **new feature vector** that is more useful for prediction than the raw inputs.

The **output layer** then uses these learned features to produce **one final number**, which is the probability that the product will be a top seller.

The key idea of this slide is simple:

**Hidden layers automatically learn better features**, which is why neural networks are powerful and reduce the need for manual feature engineering.

## Multiple hidden layers



Neural Networks might have more neurons in a hidden layer, and more than one Hidden layer

Andrew Ng

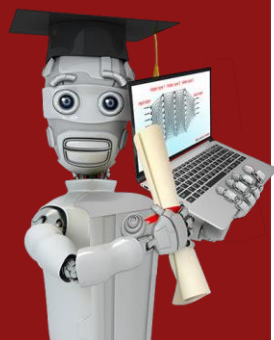
### Multiple Hidden Layers: Building Deeper Neural Networks

This slide shows how a neural network can have **more than one hidden layer** between the input and the output. A network with one hidden layer is already useful, but adding more hidden layers makes the model **deeper** and more powerful. This type of model is often called a **multilayer perceptron**.

The **input layer** receives the original data, such as numbers representing an image, sound, or other features. This input is passed to the **first hidden layer**, where neurons start to learn simple patterns from the data. Each neuron performs a small calculation and produces an activation value. The output of the first hidden layer becomes the input to the **second hidden layer**, and possibly a **third hidden layer**, and so on. Each new hidden layer learns **more abstract and higher-level patterns**. For example, in image data, early layers may detect edges, middle layers may detect shapes, and deeper layers may detect objects.

Finally, the information reaches the **output layer**, which produces the final prediction. This could be a probability (such as yes/no), a class label, or a numerical value, depending on the task.

The key idea is that **depth matters**. By stacking multiple hidden layers, neural networks can represent very complex functions. This is what allows deep learning models to solve difficult real-world problems that traditional machine learning models cannot handle easily.



# Neural Networks Intuition

## Example: Recognizing images

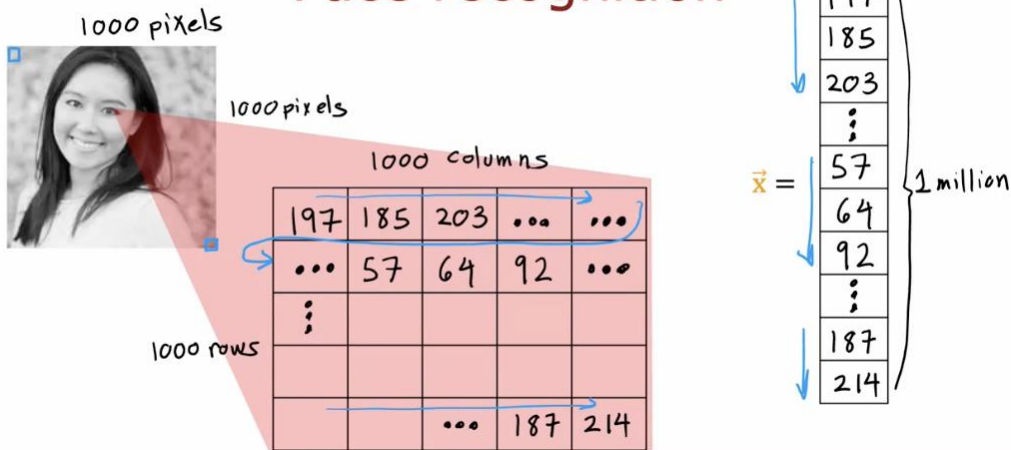
Andrew Ng

### Neural Networks Intuition: Recognizing Images

This section introduces how neural networks **recognize images**.

The goal is to build intuition for how a network can look at raw pixel values and gradually learn meaningful patterns, such as edges, shapes, and objects, to make a correct prediction.

# Face recognition



## Face Recognition: From Image to Numbers

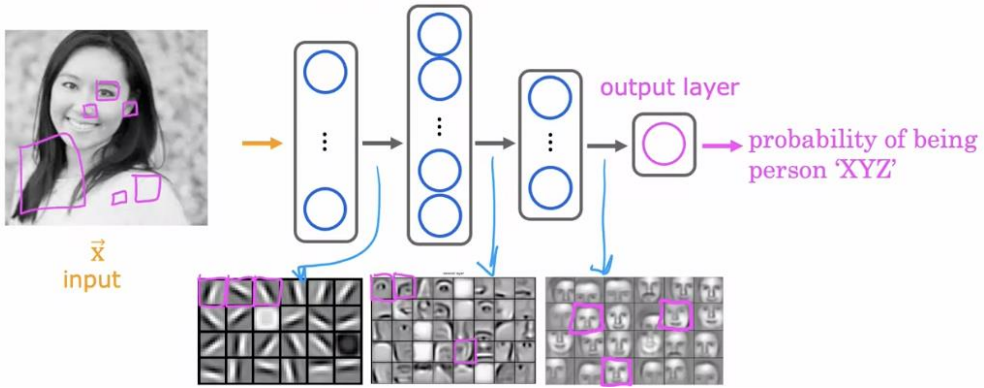
This slide explains how an **image is represented as numbers** so that a neural network can process it.

An image is made of **pixels**. In this example, the image has about **1000 rows and 1000 columns**, which means around **1 million pixels**. Each pixel is represented by a number that shows its brightness or color intensity. These pixel values are first arranged in a **matrix**, where each cell contains one number (for example, 197, 185, 203, etc.). This matrix represents the full image.

To feed the image into a neural network, the matrix is then **flattened into a single vector**. This vector is written as  $\mathbf{x}$  and contains all pixel values one after another. In this example,  $\mathbf{x}$  has about **1 million numbers**.

The key idea is that a neural network does not see a face or an image. It only sees a **large vector of numbers**. Learning face recognition means learning patterns from these numbers that correspond to facial features.

# Face recognition



A neural network recognizes faces by turning pixels into numbers and learning features step by step, from simple edges to full face patterns, until it predicts the probability of a specific person.

## Face Recognition: How Neural Networks Learn Images

This slide shows how a neural network performs **face recognition** step by step.

The process starts with the **input image**, which is converted into a large vector  $\mathbf{x}$  containing pixel values. At this stage, the network does not understand faces; it only receives **numbers**.

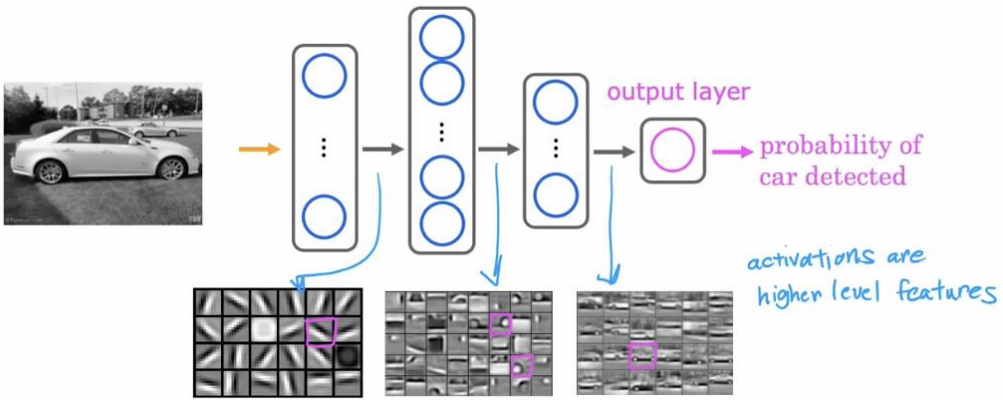
The first hidden layers learn **very simple features** from the pixels, such as edges, corners, and basic shapes. These features are small patterns found in many places in the image.

As the data moves deeper into the network, the next hidden layers combine these simple features into **more complex features**, such as eyes, noses, mouths, and facial parts.

In deeper layers, the network learns **high-level features**, such as the overall structure of a face and how facial parts are arranged together. These layers capture what makes one person's face different from another. Finally, the **output layer** produces a single value: the **probability that the image belongs to a specific person (for example, person XYZ)**.

The key idea is that neural networks recognize images by learning features **gradually**, from simple patterns to complex concepts, layer by layer.

# Car classification



A neural network classifies cars by learning visual features gradually from pixels to car parts, then using these features to predict the probability that a car is present.

## Car Classification: How Neural Networks Understand Images

This slide explains how a neural network performs **car classification** from an image. The process begins with the **input image**, which is converted into a large set of pixel values. At this stage, the network only sees numbers, not a car.

In the first hidden layers, the network learns **simple visual features** such as edges, lines, and basic textures. These features are common patterns that appear in many images and form the building blocks for understanding visual content.

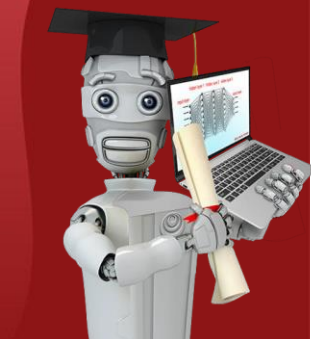
As the data moves through deeper layers, the network starts to combine simple features into **more meaningful patterns**, such as wheels, windows, and car body parts. These features are more specific and help the network distinguish cars from other objects.

In the final hidden layers, the network learns **high-level features** that represent the overall shape and structure of a car. These features capture what makes a car a car, regardless of color, background, or small variations.

The **output layer** then uses these high-level features to produce a **probability that a car is present in the image**. This probability can be compared to a threshold to make the final classification decision.

# Neural Network Model

## Neural Network Layer

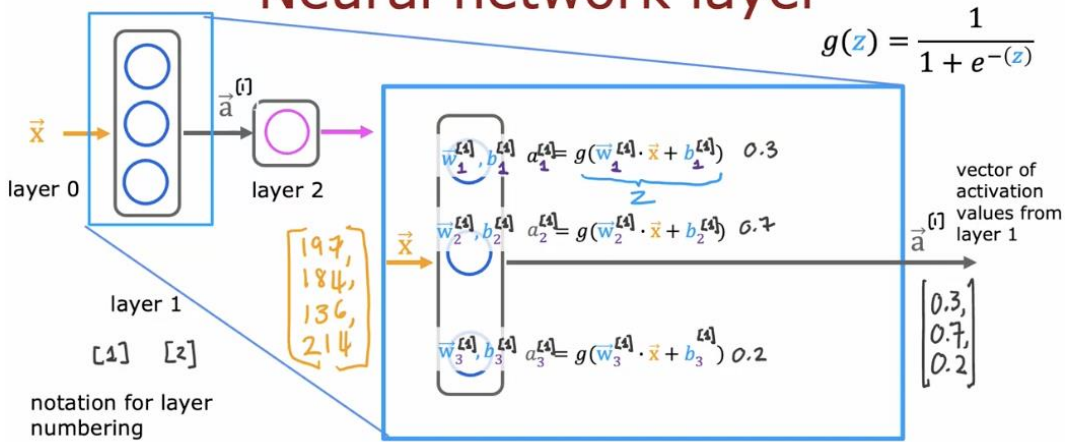


### Neural Network Model: Neural Network Layer

This section focuses on the **neural network layer**, which is the main building block of a neural network.

We will see how a layer takes inputs, applies weights and an activation function, and produces outputs that are passed to the next layer.

# Neural network layer



A neural network layer takes an input vector, applies weights, adds bias, uses an activation function, and produces a vector of activation values that is passed to the next layer. Andrew Ng

## Neural Network Layer: What Happens Inside One Layer

This slide zooms into **one hidden layer** of a neural network and explains how it works internally. A neural network is built from layers, and each layer is made of several **neurons working in parallel**. All neurons in the same layer receive the **same input vector**, but each neuron processes it differently.

The input to this layer is a vector  $\mathbf{x}$  (or  $\mathbf{a}^{(0)}$  if it comes from the previous layer). This vector contains numbers, such as pixel values in an image or feature values in a dataset. These numbers are sent to **every neuron** in the layer.

Each neuron has its own **weights ( $\mathbf{w}$ )** and **bias ( $\mathbf{b}$ )**. The neuron first computes a value called  $\mathbf{z}$ , which is a weighted sum of the inputs plus the bias:

- multiply each input by its weight
- add all the results
- add the bias

This value  $\mathbf{z}$  is not yet the output. It is just an intermediate value that measures how strong the signal is.

Next, the neuron applies an **activation function** to  $\mathbf{z}$ . In this slide, the activation function is the **sigmoid function**

$$g(z) = 1 / (1 + e^{-z}).$$

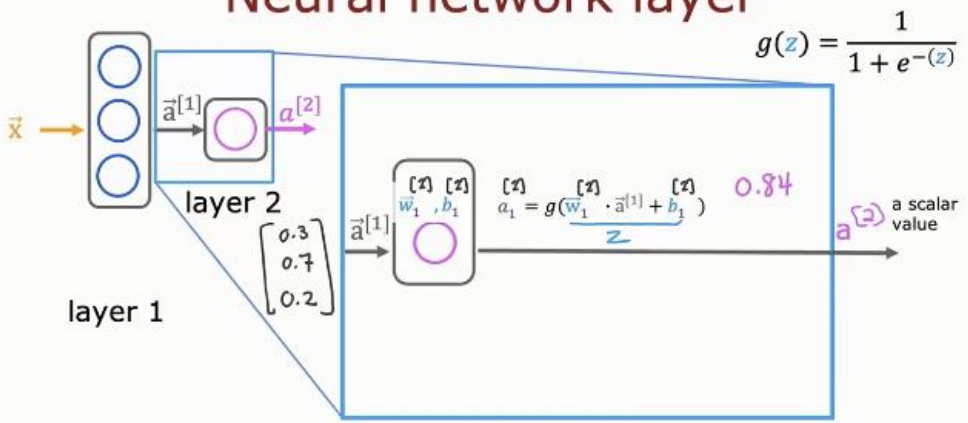
This function squashes the value into a range between **0 and 1**, which makes the neuron's output easier to interpret and helps the network learn non-linear patterns.

The result after applying the activation function is called the **activation value  $\mathbf{a}$** . Each neuron produces one activation value. All activation values from the layer are collected into a new vector  $\mathbf{a}^{(1)}$ , which becomes the **input to the next layer**.

An important idea shown here is **layer numbering**. The input layer is called **layer 0**, the first hidden layer is **layer 1**, and so on. This notation helps us

write neural networks clearly and consistently, especially when networks become deep.

# Neural network layer



A single neuron in the output layer combines inputs with weights and bias, applies the activation function, and produces one scalar value that represents the final prediction.

## Output Layer Neuron: Interpreting the Value $a[2] = 0.8$

This slide focuses on the **output layer neuron** and the value it produces, written as  $a[2]$ . Because this neuron is in the **output layer**, its activation represents the **final prediction** of the neural network.

After the input data passes through all hidden layers, the output neuron receives the activation vector from the previous layer, written as  $a[1]$ . These values are already high-level features learned by the network. The output neuron combines these values using its **weights  $w[2]$**  and **bias  $b[2]$**  to compute an intermediate value called  $z$ .

The neuron then applies the **sigmoid activation function**:

$$g(z) = 1 / (1 + e^{-(z)})$$

This function converts  $z$  into a value between **0 and 1**. The result is the activation  $a[2]$ .

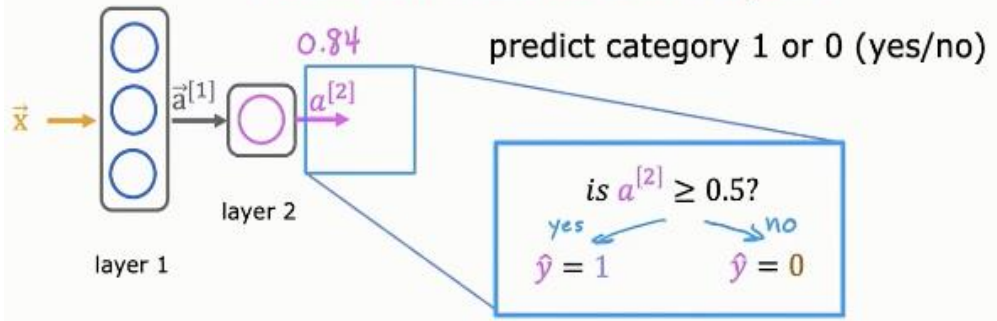
In this example, the output is:

$$a[2] = 0.8$$

This means the neural network is **80% confident** that the input belongs to the **positive class**. For example, this could mean “there is an 80% probability that the email is spam” or “an 80% chance that the image contains the target object,” depending on the application.

Since this is a **binary classification problem**, we usually use a **decision threshold**, commonly 0.5. Because  $a[2] = 0.8$  is greater than 0.5, the model predicts the **positive class (output = 1)**. If the value were below 0.5, the prediction would be the **negative class (output = 0)**.

# Neural network layer



The output layer converts the activation value into a final yes/no decision by comparing the predicted probability to a threshold (such as 0.5).

## Neural Network Output Layer: From Probability to Final Prediction

This slide explains how a neural network converts the **output layer activation** into a **final class prediction**. The output neuron produces a value called  $a^{[2]}$ , which represents the model's confidence in the positive class.

After the input  $x$  passes through the hidden layers, the network computes the output activation  $a^{[2]}$  using the sigmoid function. This value is always between **0 and 1** and is interpreted as a **probability**, not a direct class label.

In this example, the output value is:

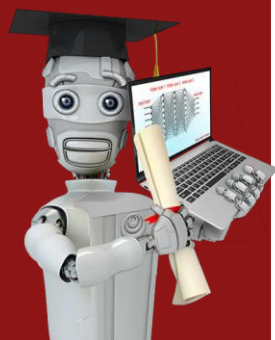
$$a^{[2]} = 0.84$$

This means the neural network estimates an **84% probability** that the input belongs to **category 1** (the "yes" or positive class).

To convert this probability into a final decision, we apply a **decision rule** using a threshold, typically **0.5**:

- If  $a^{[2]} \geq 0.5$ , then predict  $\hat{y} = 1$  (yes)
- If  $a^{[2]} < 0.5$ , then predict  $\hat{y} = 0$  (no)

Since  $a^{[2]} = 0.84$  is greater than 0.5, the network predicts **category 1**.



# Neural Network Model

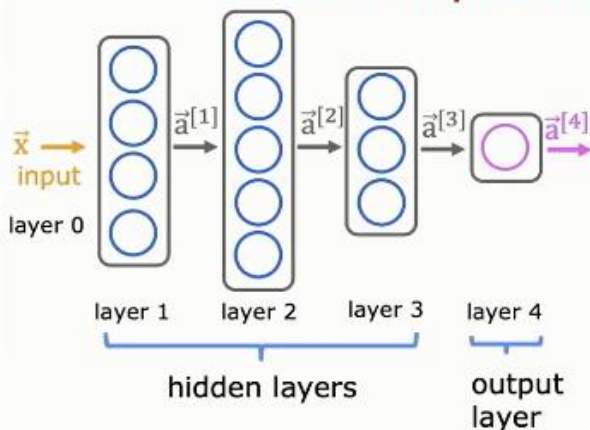
## More Complex Neural Networks

Andrew Ng

### Neural Network Model: More Complex Neural Networks

This section introduces **more complex neural networks** with multiple hidden layers, showing how stacking layers allows the model to learn richer and more abstract features.

## More complex neural network



A more complex neural network uses multiple hidden layers to transform the input step by step into higher-level features before producing the final output.

### More Complex Neural Network: Learning in Multiple Stages

This slide shows a **more complex neural network** made of several layers stacked together. The network starts with the **input layer (layer 0)**, passes through multiple **hidden layers**, and ends with a single **output layer** that produces the final prediction.

The **input layer (layer 0)** receives the raw input data  $x$ . This could be pixel values of an image, numerical features from a dataset, or any other input representation. At this stage, the network does not make decisions; it only passes information forward.

The **hidden layers (layer 1, layer 2, and layer 3)** are where learning really happens. Each hidden layer takes the output (activations) from the previous layer and transforms it into a new representation. These layers gradually learn **more complex and abstract features**. Early hidden layers usually learn simple patterns, while deeper hidden layers combine these patterns into more meaningful concepts.

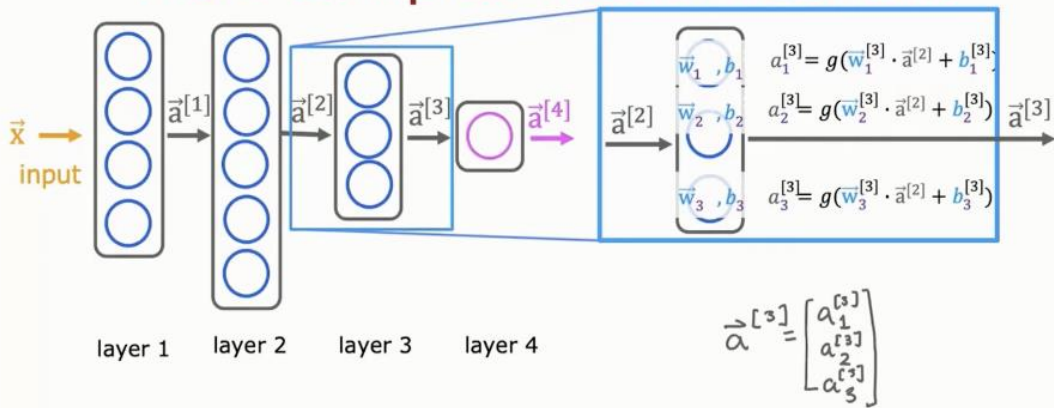
The activations are written as:

- $a[1]$  for layer 1
- $a[2]$  for layer 2
- $a[3]$  for layer 3

Each activation vector summarizes what the network has learned so far about the input.

The **output layer (layer 4)** receives the final activation vector  $a[3]$  and produces the output  $a[4]$ . This value represents the **final prediction** of the network. In binary classification,  $a[4]$  is usually a number between 0 and 1 and is interpreted as a probability.

# More complex neural network



Each hidden layer applies the same neuron computation in parallel to produce an activation vector, which is passed forward to build deeper and more powerful representations.

## More Complex Neural Network: Inside a Hidden Layer

This slide zooms into a **hidden layer inside a deep neural network** and explains how the activations of that layer are computed. The network receives the input  $x$ , passes it through several layers, and each layer transforms the information before sending it forward.

At this stage, the input to **layer 3** is the activation vector from the previous layer, written as  $\vec{a}^{[2]}$ . This vector contains values that represent features learned so far by the network. Every neuron in layer 3 receives the **same input vector  $\vec{a}^{[2]}$** , but each neuron processes it using **different weights and a different bias**.

Each neuron in layer 3 computes its own value:

$$z = w^{[3]} \cdot a^{[2]} + b^{[3]}$$

This is a weighted sum of the inputs plus a bias. The neuron then applies the **activation function  $g$** , which in this slide is the sigmoid function, to produce an activation value.

For example:

$$a_1^{[3]} = g(w_1^{[3]} \cdot a^{[2]} + b_1^{[3]})$$

$$a_2^{[3]} = g(w_2^{[3]} \cdot a^{[2]} + b_2^{[3]})$$

$$a_3^{[3]} = g(w_3^{[3]} \cdot a^{[2]} + b_3^{[3]})$$

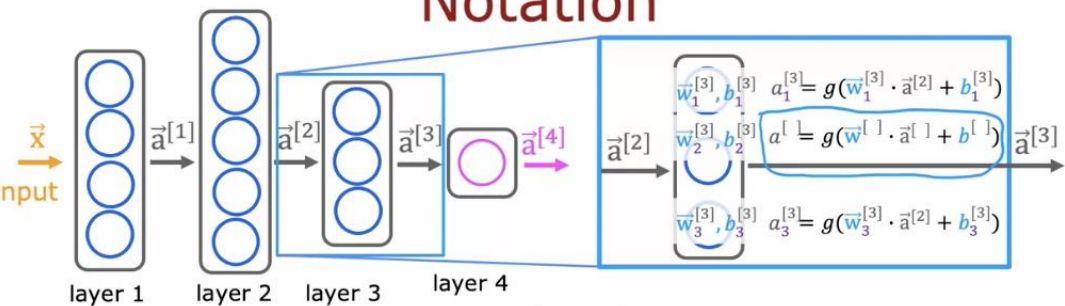
Each of these values is a **single number**, produced by one neuron in the layer.

All activation values from layer 3 are then **stacked together** to form the activation vector:

$$\vec{a}^{[3]} = [a_1^{[3]}, a_2^{[3]}, a_3^{[3]}]^T$$

This vector  $\vec{a}^{[3]}$  becomes the **input to the next layer** in the network.

# Notation



Question:

Can you fill in the superscripts and subscripts for the second neuron?

This slide summarizes neural network notation, showing how activations, weights, and biases are indexed by layer and neuron using subscripts and superscripts.

## Neural Network Notation: Understanding the Symbols

This slide explains the **notation** used to describe neural networks.

Learning this notation is very important because it allows us to describe complex neural networks in a clear, short, and consistent way. Once students understand this notation, reading formulas, slides, and code becomes much easier.

In neural networks, layers are numbered using **superscripts inside square brackets**. The input layer is called **layer 0**, and its values are written as  $\mathbf{x}$  or  $\mathbf{a}[0]$ . The first hidden layer is **layer 1**, and its output is written as  $\mathbf{a}[1]$ . The second hidden layer is **layer 2** with output  $\mathbf{a}[2]$ , and the third hidden layer is **layer 3** with output  $\mathbf{a}[3]$ . The final output layer is **layer 4**, and its output is written as  $\mathbf{a}[4]$ . In general,  $\mathbf{a}[l]$  means the activation vector produced by layer  $l$ .

Inside each layer, there are multiple neurons. To refer to a specific neuron inside a layer, we use a **subscript**. For example,  $\mathbf{a1}[3]$  means the activation of neuron 1 in layer 3, while  $\mathbf{a2}[3]$  and  $\mathbf{a3}[3]$  refer to neurons 2 and 3 in the same layer. Each neuron computes its activation by taking the activation vector from the previous layer, multiplying it by its own weights, adding a bias, and then applying an activation function.

The computation performed by a neuron in layer 3 can be written as  $a_j^{[3]} = g(\vec{w}_j^{[3]} \cdot \vec{a}^{[2]} + b_j^{[3]})$ .

Here,  $\mathbf{a}^{[2]}$  is the activation vector from the previous layer,  $\vec{w}_j^{[3]}$  represents the weights of neuron  $j$  in layer 3,  $b_j^{[3]}$  is its bias, and  $g(\cdot)$  is the activation function, such as the sigmoid function.

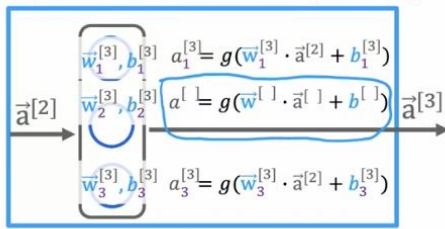
All activation values produced by neurons in the same layer are then grouped together to form an **activation vector**. For example, the activation vector of layer 3 is written as  $\mathbf{a}[3] = [a_1^{[3]}, a_2^{[3]}, a_3^{[3]}]^T$ .

This vector becomes the input to the next layer in the network.

The key idea to remember is that **superscripts indicate layer numbers**,

**subscripts indicate neuron numbers, and activation vectors flow forward from one layer to the next.** This notation allows us to describe very large and deep neural networks in a simple and organized way, and it forms the foundation for understanding forward propagation, backpropagation, and neural network implementation in code.

## [In Video Quiz]



Can you fill in the superscripts and subscripts for the second neuron?

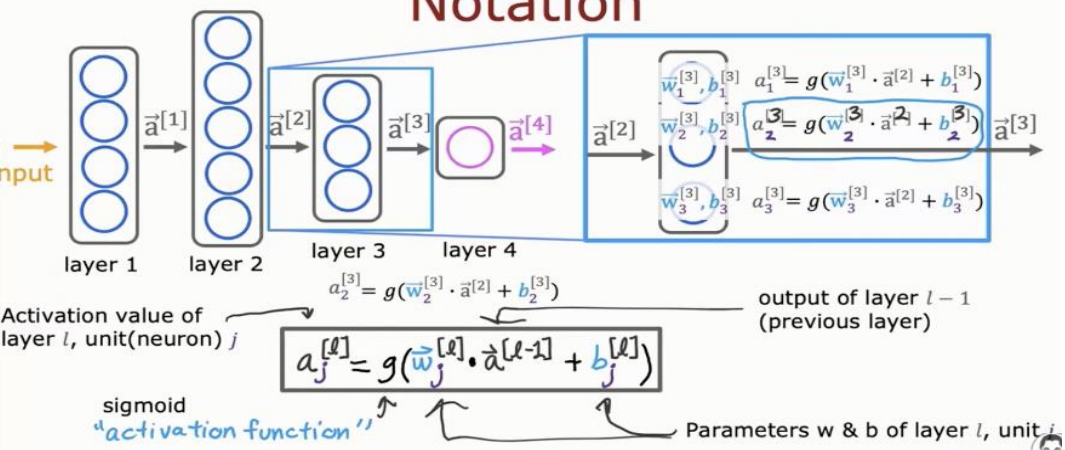
✓  $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$

✗  $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[3]} + b_2^{[3]})$  input is  $\vec{a}^{[2]}$

✗  $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot a_2^{[2]} + b_2^{[3]})$  input is a vector  $\vec{a}^{[2]}$  not a single number

This question checks whether students correctly understand neural network notation by identifying the correct layer index, neuron index, and input activation vector for the second neuron in the hidden layer.

# Notation



This slide summarizes neural network notation, showing how each neuron's activation is computed using weights, bias, and the activation function, with subscripts indicating neuron number and superscripts indicating layer number.

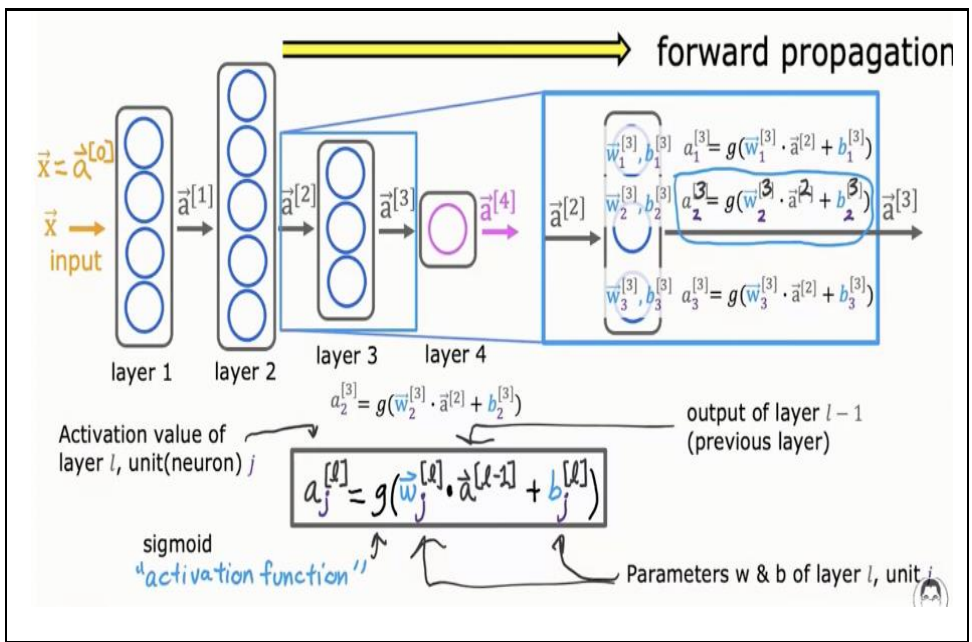
The network starts with the **input  $x$** , which is also called  $\mathbf{a}[0]$ . This input is passed to layer 1, producing the activation vector  $\mathbf{a}[1]$ . The activations then flow forward through layer 2, layer 3, and finally reach the **output layer (layer 4)**, producing  $\mathbf{a}[4]$ . In general,  $\mathbf{a}[l]$  represents the output (activation vector) of layer  $l$ , and it always comes from processing the output of the previous layer  $\mathbf{a}[l-1]$ .

Inside a layer, each neuron is identified using a **subscript  $j$** . For example,  $\mathbf{a}_j[l]$  means the activation value of **neuron  $j$  in layer  $l$** . This is a single number produced by one neuron. The slide highlights one such neuron and shows how its activation is computed.

The key formula in this slide is:

$$a_j[l] = g(w_j[l] \cdot a[l-1] + b_j[l])$$

This formula tells us exactly what one neuron does. First, the neuron takes the activation vector from the **previous layer**, written as  $\mathbf{a}[l-1]$ . It multiplies this vector by its own **weight vector  $w_j[l]$** , then adds a **bias term  $b_j[l]$** . This produces an intermediate value called  $\mathbf{z}$ . Next, the neuron applies an **activation function  $g(\cdot)$** , which in this slide is the **sigmoid function**. The output of this function is the activation value  $\mathbf{a}_j[l]$ .



### Forward Propagation: A Brief Explanation

Forward propagation is the process by which a neural network **computes its output** by moving information **from the input layer to the output layer**. It is the step the network uses to **make a prediction**.

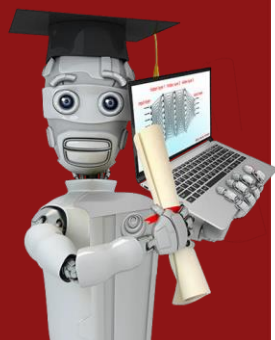
The process starts with the input  $x$ , also written as  $a[0]$ . This input is passed to the first hidden layer, where each neuron computes an activation using weights, a bias, and an activation function. This produces the activation vector  $a[1]$ .

The same computation is repeated layer by layer. The output of each layer  $a[l-1]$  becomes the input to the next layer  $a[l]$ , using the formula:

$$a_j[l] = g(w_j[l] \cdot a[l-1] + b_j[l])$$

This continues until the output layer produces  $a[L]$ , which is the **final prediction** of the network. In classification, this value is often interpreted as a probability.

Forward propagation always moves in **one direction only** and does **not change any parameters**. It simply applies the current weights and biases to compute the output.



# Neural Network Model

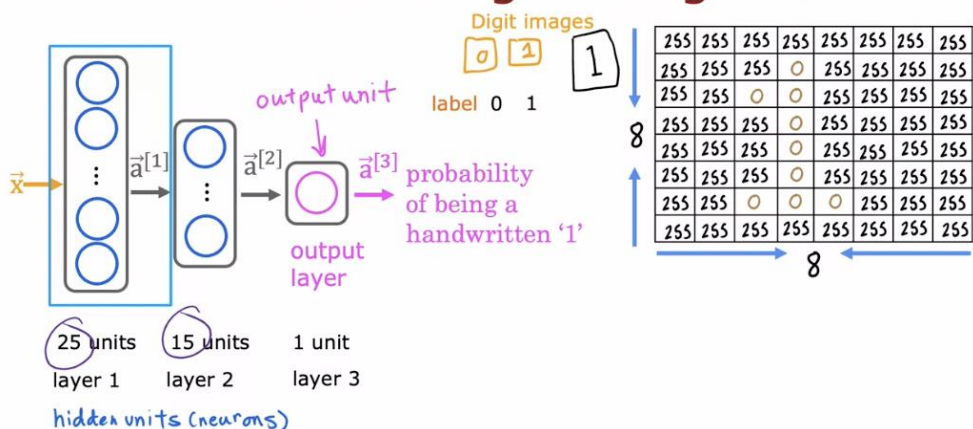
## Inference: Making Predictions (forward Propagation)

Andrew Ng

### Neural Network Model: Inference (Making Predictions)

This section explains **inference**, which is the process of making predictions using a neural network through **forward propagation**, from input to output.

# Handwritten digit recognition



## Handwritten Digit Recognition: Neural Network Inference

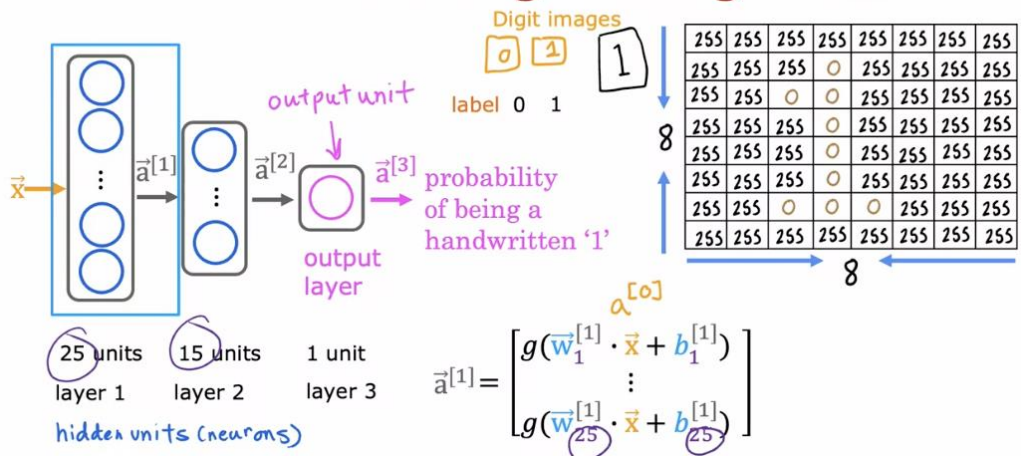
The input to the neural network is a **digit image**, represented as an **8 × 8 grid of pixel values** (numbers from 0 to 255). These pixel values are flattened into a single input vector  $\mathbf{x}$ .

The network has **two hidden layers** with **25** and **15 neurons**. These layers process the pixel values and learn useful patterns related to handwritten digits.

The **output layer** has **one neuron** that produces a value  $\mathbf{a}[3]$ , which represents the **probability that the image is the handwritten digit "1"**.

During inference, the input is passed forward through the network using the learned weights and biases. A threshold (such as 0.5) is used to convert the probability into the final prediction.

# Handwritten digit recognition



## Handwritten Digit Recognition: Key Idea

This slide shows how a neural network recognizes a **handwritten digit** using **forward propagation**.

The input is an **8 × 8 digit image**, where each pixel is a number between **0 and 255**. These pixel values are flattened into one input vector  $\mathbf{x} = \mathbf{a}^{[0]}$ .

The network has **two hidden layers**:

- **Layer 1** with **25 neurons**
- **Layer 2** with **15 neurons**

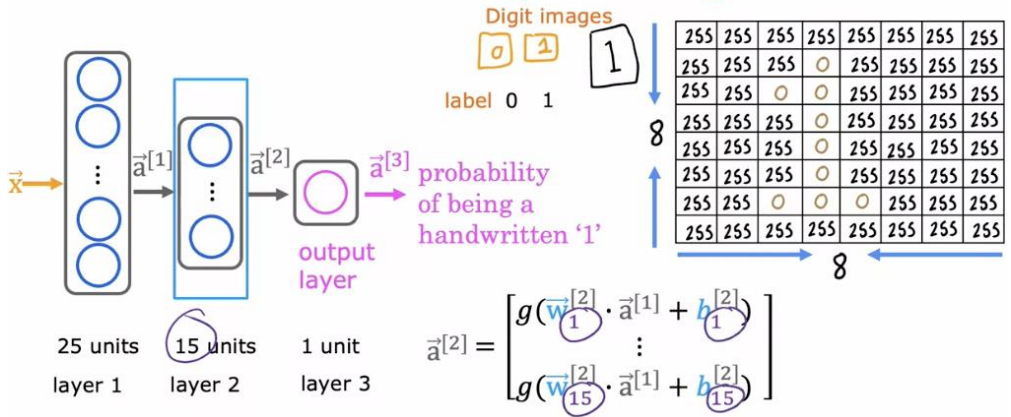
Each hidden layer computes activation values using weights, bias, and an activation function, learning patterns such as strokes and shapes in the digit.

The **output layer (layer 3)** has **one neuron** that produces  $\mathbf{a}^{[3]}$ , which represents the **probability that the image is the digit "1"**.

This probability is then used to decide whether the digit is **1 or not**.

**Key point:** raw pixel values → hidden layers learn features → output gives a probability prediction.

# Handwritten digit recognition



## Handwritten Digit Recognition: What This Slide Shows

This slide explains how a neural network predicts whether an image is the handwritten digit “1”.

The input is an **8 × 8 image** of a digit. Each pixel has a value between **0 and 255**, and all pixel values are flattened into one input vector  $\mathbf{x}$ .

The network has **two hidden layers**:

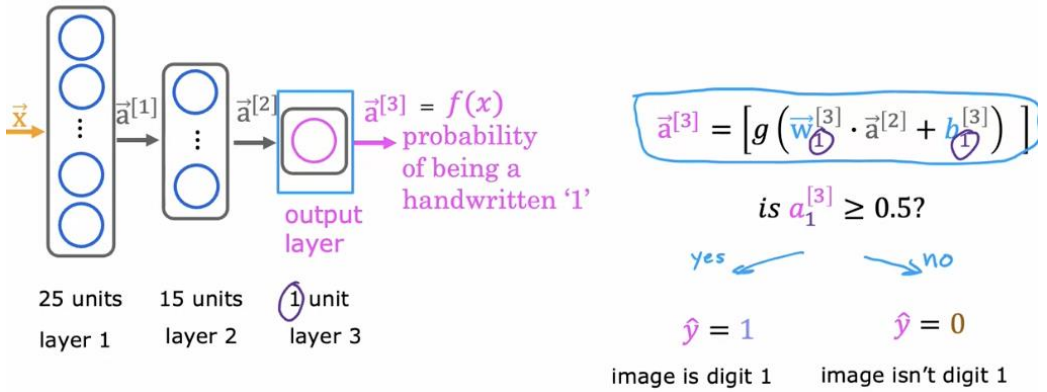
- **Layer 1** with **25 neurons**
- **Layer 2** with **15 neurons**

Each hidden layer takes the activations from the previous layer and computes new activation values using **weights, bias, and an activation function**. These layers learn important patterns in the digit, such as strokes and shapes.

The **output layer (layer 3)** has **one neuron**. It produces  $\mathbf{a}^{[3]}$ , which represents the **probability that the image is the handwritten digit “1”**.

**Key idea:** the network transforms pixel values step by step through hidden layers, and the output layer gives a probability used to make the final prediction.

# Handwritten digit recognition



## Handwritten Digit Recognition: Final Decision

This slide shows how the neural network makes the **final prediction** for handwritten digit recognition.

The input image is processed through **two hidden layers** (25 neurons, then 15 neurons). Each layer computes activation values that capture patterns in the image.

The **output layer** has **one neuron**. It computes  $a[3] = f(x)$ ,

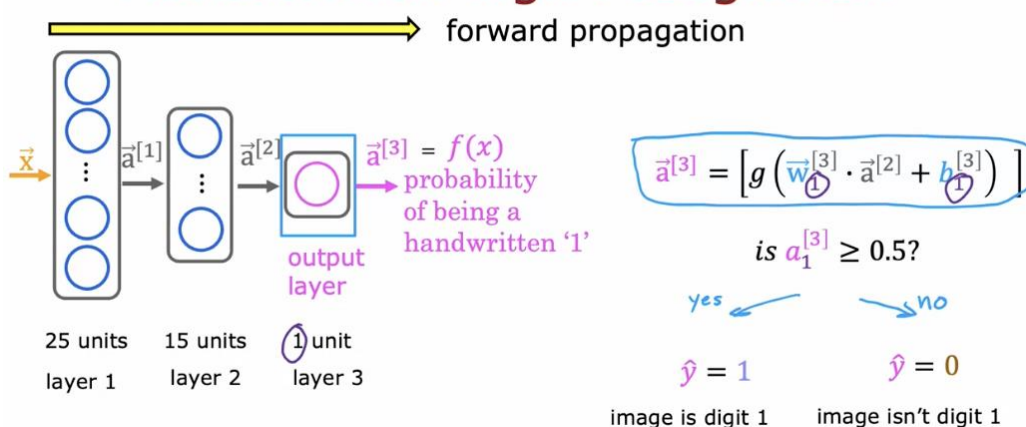
which represents the **probability that the image is the digit "1"**.

To make a clear decision, this probability is compared to a threshold:

- If  $a[3] \geq 0.5 \rightarrow$  predict  $\hat{y} = 1$  (the image is digit 1)
- If  $a[3] < 0.5 \rightarrow$  predict  $\hat{y} = 0$  (the image is not digit 1)

**Key idea:** the network turns an image into a probability, then uses a simple threshold to decide yes or no.

# Handwritten digit recognition



## Handwritten Digit Recognition: Forward Propagation

This slide shows **forward propagation** for handwritten digit recognition. The input image is passed through the network **from left to right**, layer by layer.

The image pixels form the input  $x$ , which is processed by **layer 1 (25 neurons)** and then **layer 2 (15 neurons)**. Each layer computes activation values using weights, bias, and an activation function.

The **output layer (1 neuron)** computes

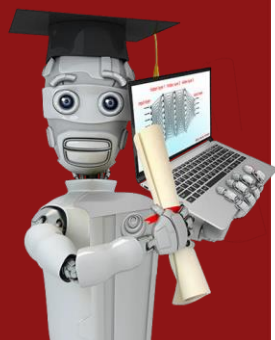
$$a[3] = f(x),$$

which represents the **probability that the image is the handwritten digit "1"**.

This probability is then compared to a threshold:

- If  $a[3] \geq 0.5 \rightarrow$  predict  $\hat{y} = 1$  (image is digit 1)
- If  $a[3] < 0.5 \rightarrow$  predict  $\hat{y} = 0$  (image is not digit 1)

**Key idea:** forward propagation takes the input image and produces a probability, which is then converted into a final yes/no prediction.



# TensorFlow Implementation

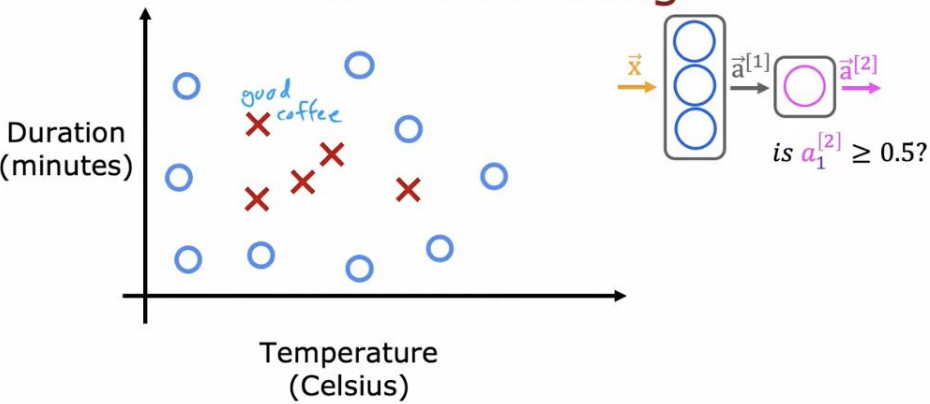
## Inference in Code

Andrew Ng

### TensorFlow Implementation: Inference in Code

This section shows how **forward propagation (inference)** is implemented in TensorFlow code to make predictions using a trained neural network.

# Coffee roasting



## Coffee Roasting: Neural Network Inference Example

This slide uses **coffee roasting** as a simple real-world example to explain **inference (making predictions)** with a neural network.

Each data point represents one coffee roast. The **input features** are:

- **Temperature (Celsius)**
- **Duration (minutes)**

Together, these two values form the input vector  $\mathbf{x}$ .

The points are labeled as:

- **Good coffee**
- **Bad coffee**

The goal of the neural network is to learn from past roasting examples and then **predict whether a new roast will be good or not**.

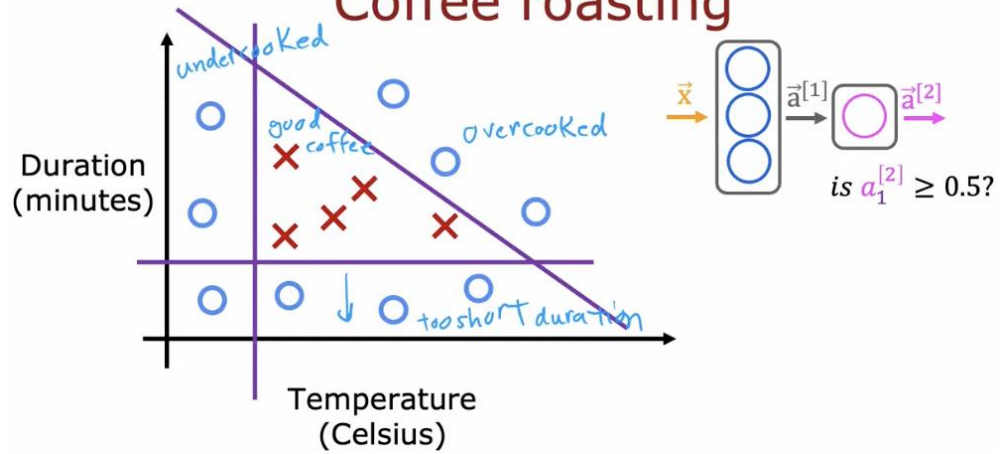
The input vector  $\mathbf{x}$  is passed into a small neural network. The network computes an output value  $\mathbf{a}^{[2]}$ , which represents the **probability that the coffee roast is good**.

To make a final decision, the network applies a threshold:

- If  $\mathbf{a}^{[2]} \geq 0.5$ , predict **good coffee**
- If  $\mathbf{a}^{[2]} < 0.5$ , predict **bad coffee**

**Key idea:** the neural network learns a decision boundary based on temperature and time, and then uses forward propagation to predict coffee quality.

# Coffee roasting



## Coffee Roasting: Decision Boundary and Prediction

This slide shows how a neural network learns a **decision boundary** for the coffee roasting problem.

Each point represents one coffee roast, described by two inputs:

- **Temperature (Celsius)**
- **Duration (minutes)**

Roasts can be **good coffee** or **bad coffee**. Bad coffee may happen because the roast is **undercooked** (low temperature or short time) or **overcooked** (high temperature or long time).

The diagonal line represents the **decision boundary** learned by the model. This boundary separates regions where the network predicts good coffee from regions where it predicts bad coffee.

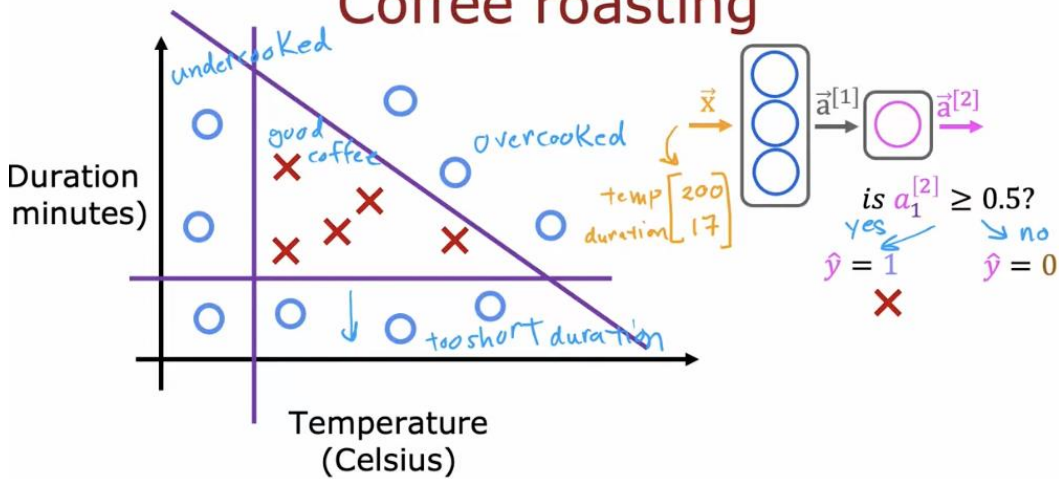
For a new roast, the input vector  $\mathbf{x}$  (temperature and duration) is passed through the neural network. The network produces  $\mathbf{a}^{[2]}$ , which is the **probability that the coffee is good**.

The final decision is made using a threshold:

- If  $\mathbf{a}^{[2]} \geq 0.5$ , predict **good coffee**
- If  $\mathbf{a}^{[2]} < 0.5$ , predict **bad coffee**

**Key idea:** the neural network learns how temperature and time together affect coffee quality and uses a learned decision boundary to make predictions.

# Coffee roasting



DeepLearning.AI Stanford ONLINE

## Coffee Roasting: Making a Prediction with a Neural Network

This slide shows how a neural network makes a **prediction** for one specific coffee roast using **forward propagation** and a **decision boundary**.

Each coffee roast is described by two input features:

- **Temperature (Celsius)**
- **Duration (minutes)**

From the data, we see three main regions:

- **Undercooked** coffee (low temperature or short time)
- **Overcooked** coffee (high temperature or long time)
- **Good coffee** in the middle region

The diagonal line represents the **decision boundary** learned by the neural network. This boundary separates the region where coffee is predicted to be good from the region where it is predicted to be bad.

For a new roast, the input vector  $\mathbf{x}$  (for example, temperature = 200°C and duration = 17 minutes) is fed into the neural network. The network computes an output value  $\mathbf{a}^{[2]}$ , which represents the **probability that the coffee is good**.

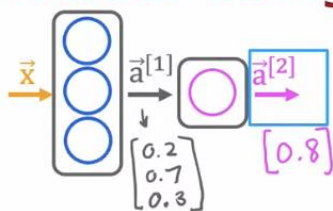
The final decision is made using a threshold:

- If  $\mathbf{a}^{[2]} \geq 0.5$ , then  $\hat{\mathbf{y}} = 1 \rightarrow$  good coffee
- If  $\mathbf{a}^{[2]} < 0.5$ , then  $\hat{\mathbf{y}} = 0 \rightarrow$  bad coffee

In this example, the point falls in the **good coffee region**, so the network predicts **good coffee**.

**Key idea:** the neural network combines temperature and time, learns a decision boundary, and uses it to predict whether a coffee roast will be good or bad.

# Build the model using TensorFlow



```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

is  $a_1^{[2]} \geq 0.5$ ?  
yes  $\hat{y} = 1$  (marked with a red X)  
no  $\hat{y} = 0$  (marked with a blue circle)

```
if a2 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```

## Build the Model Using TensorFlow (Inference Example)

This slide shows how to build the same small neural network in **TensorFlow** and use it to make a prediction.

The input  $\mathbf{x}$  is one example with two features: **temperature and duration**, written as:

$\mathbf{x} = [200.0, 17.0]$

The model has two layers:

- **Layer 1:** `Dense(units=3, activation='sigmoid')`

This layer takes  $\mathbf{x}$  and produces  $\mathbf{a1}$ , a vector of 3 activation values (for example  $[0.2, 0.7, 0.3]$ ).

- **Layer 2:** `Dense(units=1, activation='sigmoid')`

This layer takes  $\mathbf{a1}$  and produces  $\mathbf{a2}$ , a single value (for example 0.8).

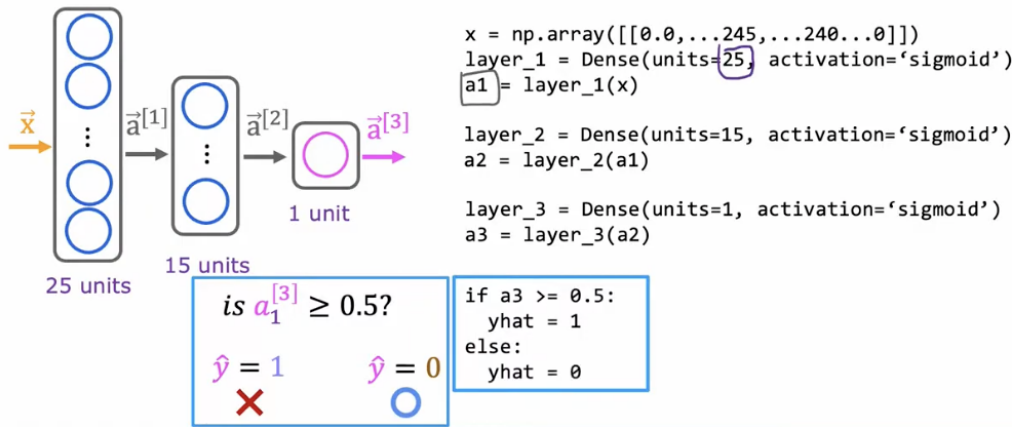
The output  $\mathbf{a2}$  is interpreted as the **probability of good coffee**. To convert it into a final decision, we use a threshold:

- If  $\mathbf{a2} \geq 0.5 \rightarrow \hat{y} = 1$  (good coffee)

- Else  $\rightarrow \hat{y} = 0$  (bad coffee)

**Key idea:** TensorFlow layers compute activations step by step ( $\mathbf{x} \rightarrow \mathbf{a1} \rightarrow \mathbf{a2}$ ), and then we use a threshold to get the final class prediction.

# Model for digit classification



## Model for Digit Classification (Inference Only)

This slide shows a **neural network model** used to classify handwritten **digits**, focusing on **forward propagation (inference)**.

The input  $\mathbf{x}$  is a vector of pixel values from a digit image (for example, 28×28 pixels flattened into one vector).

The network has **three layers**:

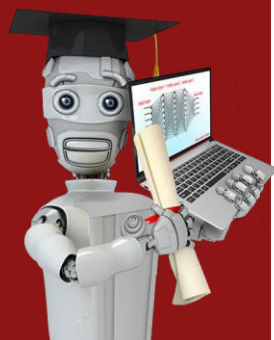
- **Layer 1 (25 units)**: takes the image pixels and computes activations  $\mathbf{a1}$  using a sigmoid function.
- **Layer 2 (15 units)**: takes  $\mathbf{a1}$  and computes new activations  $\mathbf{a2}$ .
- **Layer 3 (1 unit – output layer)**: takes  $\mathbf{a2}$  and produces  $\mathbf{a3}$ , a single value.

The output  $\mathbf{a3}$  represents the **probability that the image belongs to a specific digit class** (for example, digit “1”).

To make the final prediction:

- If  $\mathbf{a3} \geq 0.5$ , predict  $\mathbf{y\_hat} = 1$  (digit detected)
- If  $\mathbf{a3} < 0.5$ , predict  $\mathbf{y\_hat} = 0$  (digit not detected)

**Key idea:** the image is passed layer by layer through the network, and the final neuron outputs a probability that is converted into a yes/no classification.



# TensorFlow Implementation

## Data in TensorFlow

Andrew Ng

### TensorFlow Implementation: Data in TensorFlow

This section explains how **input data, labels, and parameters** are stored in TensorFlow as **tensors** (multi-dimensional arrays).

It also shows how data shapes must match the model layers so TensorFlow can perform computations correctly during training and inference.

# Feature vectors

| temperature<br>(Celsius) | duration<br>(minutes) | Good coffee?<br>(1/0) |
|--------------------------|-----------------------|-----------------------|
| 200.0                    | 17.0                  | 1                     |
| 425.0                    | 18.5                  | 0                     |
| ...                      | ...                   | ...                   |

`x = np.array([[200.0, 17.0]])` ←  
`[[200.0, 17.0]]`  
*why?*

## Feature Vectors (Why this format?)

Each training example is described by **multiple features**, here:

- temperature (200.0)
- duration (17.0)

These values are grouped into **one feature vector**:

`x = [200.0, 17.0]`

In TensorFlow, we write it as:

`x = np.array([[200.0, 17.0]])`

because:

- the **inner brackets** `[200.0, 17.0]` represent **one example**
- the **outer brackets** `[[ ... ]]` represent a **batch of examples**

**Key idea:** even one data point must be written as a 2D array so it matches TensorFlow's expected input shape.

# Note about numpy arrays

3 columns  
2 rows  
 $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$   
2 x 3 matrix

```
x = np.array([[1, 2, 3],  
              [4, 5, 6]])  
[[1, 2, 3],  
 [4, 5, 6]]
```

2D array

2 x 3

4 x 2

1 x 2

2 x 1

4 rows  
 $\begin{bmatrix} 0.1 & 0.2 \\ -3 & -4 \\ -0.5 & -0.6 \\ 7 & 8 \end{bmatrix}$   
2 columns  
4 x 2 matrix

```
x = np.array([[0.1, 0.2],  
              [-3.0, -4.0],  
              [-0.5, -0.6],  
              [7.0, 8.0]])  
[[0.1, 0.2],  
 [-3.0, -4.0],  
 [-0.5, -0.6],  
 [7.0, 8.0]]
```

## Note about NumPy Arrays (Simple Explanation)

A **NumPy array** stores data in rows and columns, like a table (matrix).

```
• x = np.array([[1, 2, 3], [4, 5, 6]])
```

This is a **2D array** with **2 rows and 3 columns** → shape **(2 x 3)**.

- Each **inner list** is one row.

- The number of values in each row = number of columns.

Another example:

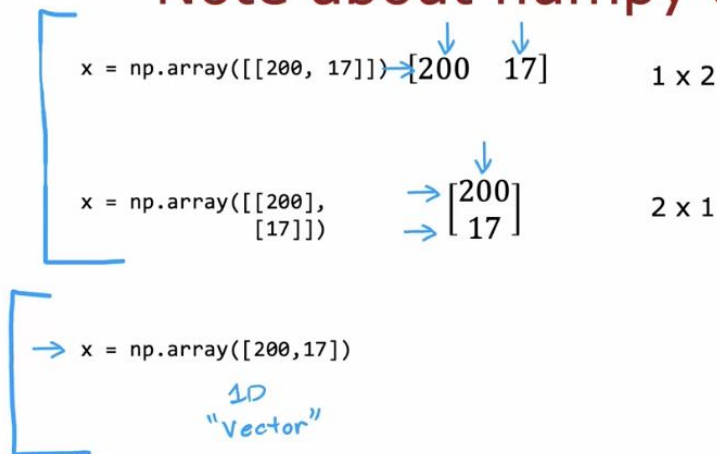
```
• x = np.array([[0.1, 0.2], [-3.0, -4.0], [-0.5, -0.6], [7.0, 8.0]])
```

This is **4 rows and 2 columns** → shape **(4 x 2)**.

### Key idea:

NumPy arrays must have a clear shape (rows x columns), and this shape is very important when working with TensorFlow and neural networks.

# Note about numpy arrays



## Note about NumPy Arrays

In NumPy, the **way we write brackets determines the shape of the array**, and the shape is very important when working with neural networks.

When we write

```
x = np.array([[200, 17]])
```

this creates a **2D array with shape (1 x 2)**. It means **one example (one row) with two features**. This is the correct and expected format when we give a single input example to a neural network in TensorFlow.

When we write

```
x = np.array([[200], [17]])
```

this creates a **2D array with shape (2 x 1)**. It means **two rows and one column**, which is **not the same** as one example with two features. Even though the numbers are the same, the meaning is different, and this format is usually not suitable as model input.

When we write

```
x = np.array([200, 17])
```

this creates a **1D array (a vector)** with shape (2,). This array has **no row-column structure**, so TensorFlow often cannot use it directly as input to a neural network.

The key idea is that **neural networks expect input data in 2D form**: (number of examples, number of features).

So for one example with two features, the correct format is always:

```
x = np.array([[200, 17]])
```

# Feature vectors

| temperature<br>(Celsius) | duration<br>(minutes) | Good coffee?<br>(1/0) |
|--------------------------|-----------------------|-----------------------|
| 200.0                    | 17.0                  | 1                     |
| 425.0                    | 18.5                  | 0                     |
| ...                      | ...                   | ...                   |

```
x = np.array([[200.0, 17.0]])
```

←

```
[[200.0, 17.0]]
```

↓ ↓

→ [200.0 17.0]

1 x 2

## Feature Vectors

Each training example is described by **input features**. In this example, the features are **temperature (Celsius)** and **duration (minutes)**, and the label tells us whether the coffee is good (1) or not (0).

For one example, the feature values are **200.0** and **17.0**. These two numbers are grouped together to form a **feature vector**, written as: [200.0, 17.0].

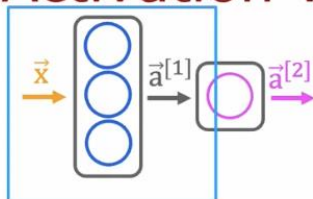
In NumPy and TensorFlow, we must represent this feature vector as a **2D array**:

```
x = np.array([[200.0, 17.0]]).
```

The inner brackets [200.0, 17.0] represent **one example**, and the outer brackets [[ ... ]] represent a **batch of examples**. The resulting shape is **1 x 2**, meaning **one example with two features**.

The key idea is that **each row is one data example**, and **each column is one feature**. This format allows the neural network to correctly process the input data.

# Activation vector



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
→ [[0.2, 0.7, 0.3]] 1 x 3 matrix
   tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)
→ a1.numpy()
   array([[0.2, 0.7, 0.3]], dtype=float32)
```

## Activation Vector

After the input feature vector  $x$  is given to a neural network layer, the layer computes **activation values**. These values are the **outputs of the neurons in that layer**.

In this example, the input is:

```
x = np.array([[200.0, 17.0]])
```

This is one example with two features.

The first layer is defined as:

```
layer_1 = Dense(units=3, activation='sigmoid')
```

This means the layer has **3 neurons**, so it will produce **3 activation values**.

When we apply the layer:

```
a1 = layer_1(x)
```

the result is an **activation vector**:

```
[[0.2, 0.7, 0.3]]
```

This vector has shape **(1 × 3)**, which means:

- **1 row** → one input example
- **3 columns** → one activation value from each neuron in the layer

In TensorFlow, this result is stored as a **tensor**:

```
tf.Tensor([[0.2, 0.7, 0.3]], shape=(1, 3))
```

When we call:

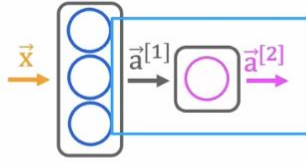
```
a1.numpy()
```

we convert the tensor into a **NumPy array**:

```
array([[0.2, 0.7, 0.3]])
```

The key idea is that **the activation vector represents what the network has learned from the input at this layer**. Each number shows how strongly one neuron is activated, and this vector is then passed to the next layer.

# Activation vector



```
> layer_2 = Dense(units=1, activation='sigmoid')
> a2 = layer_2(a1)
    ↪  $[[0.8]]$  ←  $1 \times 1$ 
> tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
> a2.numpy()
    ↪ array([[0.8]], dtype=float32)
```

## Activation Vector (Output Layer)

In this slide, we look at the **activation vector of the output layer**.

The second layer is defined as:

```
layer_2 = Dense(units=1, activation='sigmoid')
```

This means the layer has **one neuron**, so it produces **one activation value**.

When we apply this layer to the previous activation vector **a1**:

```
a2 = layer_2(a1)
```

the result is:

```
[[0.8]]
```

This output has shape **(1 × 1)**, which means:

- **1 row** → one input example
- **1 column** → one output neuron

In TensorFlow, this value is stored as a **tensor**:

```
tf.Tensor([[0.8]], shape=(1, 1))
```

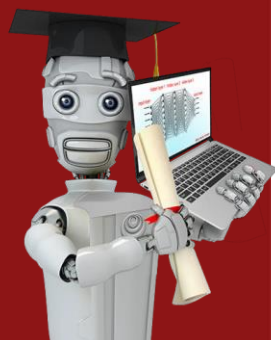
When we call:

```
a2.numpy()
```

it becomes a NumPy array:

```
array([[0.8]])
```

The key idea is that **the activation vector of the output layer contains the final prediction**. In this example, the value **0.8** represents the **probability of the positive class**, which is later compared to a threshold (such as 0.5) to make a final decision.



# TensorFlow Implementation

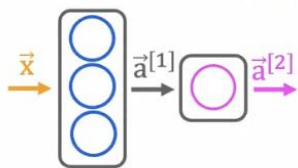
## Building a Neural Network

Andrew Ng

### **TensorFlow Implementation: Building a Neural Network**

This section shows how to **build a neural network step by step in TensorFlow**, by defining layers, choosing activation functions, and connecting the layers to form a complete model.

# What you saw earlier



```
→ x = np.array([[200.0, 17.0]])  
→ layer_1 = Dense(units=3, activation="sigmoid")  
→ a1 = layer_1(x)  
  
→ layer_2 = Dense(units=1, activation="sigmoid")  
→ a2 = layer_2(a1)
```

## What You Saw Earlier

Earlier, you saw how **forward propagation** is written directly in TensorFlow code.

First, the input feature vector  $\mathbf{x}$  is created as a NumPy array.

Then, **Layer 1** is defined with 3 neurons and a sigmoid activation, and applying it to  $\mathbf{x}$  produces the activation vector  $\mathbf{a1}$ .

Next, **Layer 2** is defined with 1 neuron and a sigmoid activation, and applying it to  $\mathbf{a1}$  produces  $\mathbf{a2}$ , the final output.

This code is a direct implementation of the neural network diagram:

$\mathbf{x} \rightarrow \mathbf{a1} \rightarrow \mathbf{a2}$ .

# Building a neural network architecture

```
→ layer_1 = Dense(units=3, activation="sigmoid") ←
→ layer_2 = Dense(units=1, activation="sigmoid") ←
→ model = Sequential([layer_1, layer_2])

x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
# 4 x 2

# targets
y = np.array([1,0,0,1])

model.compile(...) ← more about this next week!
model.fit(x,y)
→ model.predict(x_new) ←
```

|     |    | y |
|-----|----|---|
| 200 | 17 | 1 |
| 120 | 5  | 0 |
| 425 | 20 | 0 |
| 212 | 18 | 1 |

DeepLearning.AI Stanford ONLINE

## Building a Neural Network Architecture

This slide shows how we move from **individual layers** to a **complete neural network model** in TensorFlow.

First, we define the layers of the network.

`layer_1 = Dense(units=3, activation="sigmoid")` creates a hidden layer with 3 neurons.

`layer_2 = Dense(units=1, activation="sigmoid")` creates the output layer with 1 neuron.

These layers match the neural network diagram on the left.

Next, we connect the layers together using:

`model = Sequential([layer_1, layer_2])`.

The `Sequential` model means the data flows **in order**, from layer 1 to layer 2, exactly like forward propagation.

The input data **x** is a NumPy array with shape **(4 × 2)**.

This means we have **4 training examples**, and each example has **2 features** (for example, temperature and duration).

The target labels **y** are stored in a separate array, where each value (0 or 1) is the correct answer for one example.

After defining the model and data, we prepare the model using:

`model.compile(...)`.

This step tells TensorFlow how the model should learn (details will be explained later).

Then we train the model using:

`model.fit(x, y)`.

During training, TensorFlow adjusts the weights so the model's predictions get closer to the target values.

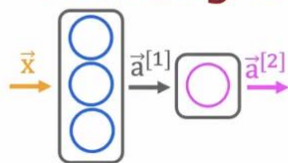
Finally, once the model is trained, we can make predictions on new data using:

`model.predict(x_new)`.

The key idea is that **TensorFlow lets us build a neural network by defining**

**layers, stacking them into a model, training with data, and then using the model to make predictions.**

# Building a neural network architecture



```
→ model = Sequential([  
→   Dense(units=3, activation="sigmoid"),  
→   Dense(units=1, activation="sigmoid")])
```

|     |    | y |
|-----|----|---|
| 200 | 17 | 1 |
| 120 | 5  | 0 |
| 425 | 20 | 0 |
| 212 | 18 | 1 |

```
x = np.array([[200.0, 17.0],  
              [120.0, 5.0],  
              [425.0, 20.0],  
              [212.0, 18.0]])
```

4 x 2

*targets* `y = np.array([1,0,0,1])`

```
model.compile(...)
```

```
model.fit(x,y)
```

```
→ model.predict(x_new) ←
```

*← more about this next week!*

## Alternative Way of defining layers

DeepLearning.AI Stanford ONLINE

### Building a Neural Network Architecture (Alternative Way)

This slide shows an **alternative and common way** to build a neural network in TensorFlow using the **Sequential** model.

Instead of defining each layer separately, we define the whole model at once:

```
model = Sequential([ Dense(...), Dense(...) ]).
```

Here, the first Dense layer has **3 neurons** with a sigmoid activation (hidden layer), and the second Dense layer has **1 neuron** with a sigmoid activation (output layer). The data flows through these layers **in order**.

The input data **x** is a NumPy array with shape **(4 × 2)**.

This means there are **4 training examples**, and each example has **2 features** (for example, temperature and duration).

The target labels **y** are stored in a separate array:

```
y = [1, 0, 0, 1].
```

Each value in **y** is the correct answer for the corresponding row in **x**.

After defining the model, we prepare it using:

```
model.compile(...).
```

This step sets how the model will learn (details come later).

Then we train the model using:

```
model.fit(x, y).
```

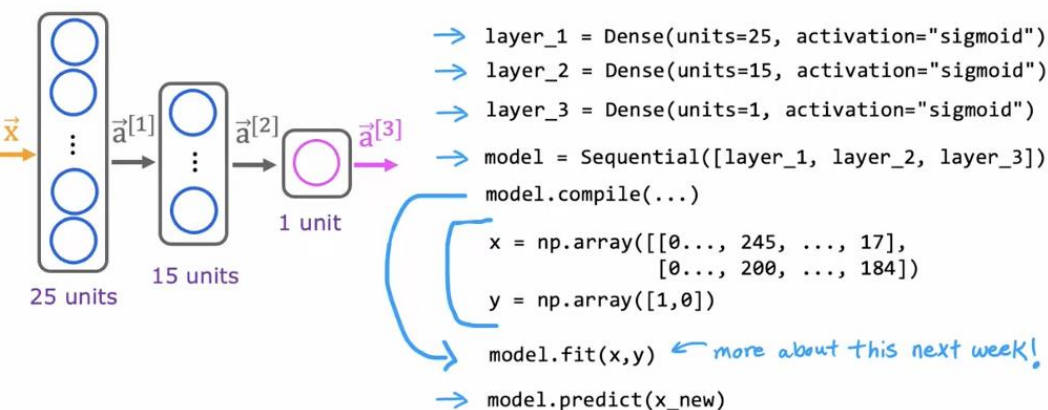
During training, TensorFlow adjusts the weights to match predictions with the targets.

Finally, we can use the trained model to make predictions on new data with:

```
model.predict(x_new).
```

The key idea is that **Sequential provides a clean and simple way to define, train, and use a neural network in TensorFlow**.

# Digit classification model



## Digit Classification Model

This slide shows a **neural network model for digit classification**, built using TensorFlow.

The input  $\mathbf{x}$  represents digit images after preprocessing (for example, pixel values flattened into a vector). Each row in  $\mathbf{x}$  is one image. The target  $\mathbf{y}$  contains the correct labels (such as 0 or 1) for each image.

The model has **three layers**:

•**Layer 1:** `Dense(units=25, activation="sigmoid")`

This is a hidden layer with 25 neurons that learns low-level patterns from the image.

•**Layer 2:** `Dense(units=15, activation="sigmoid")`

This hidden layer learns higher-level features based on the first layer.

•**Layer 3:** `Dense(units=1, activation="sigmoid")`

This is the output layer and produces  $\mathbf{a}^{[3]}$ , a single value representing the **probability of the digit belonging to a certain class**.

The layers are combined into one model using:

```
model = Sequential([layer_1, layer_2, layer_3]).
```

After defining the model, we call:

```
model.compile(...)
```

to prepare the model for learning.

Then the model is trained using:

```
model.fit(x, y),
```

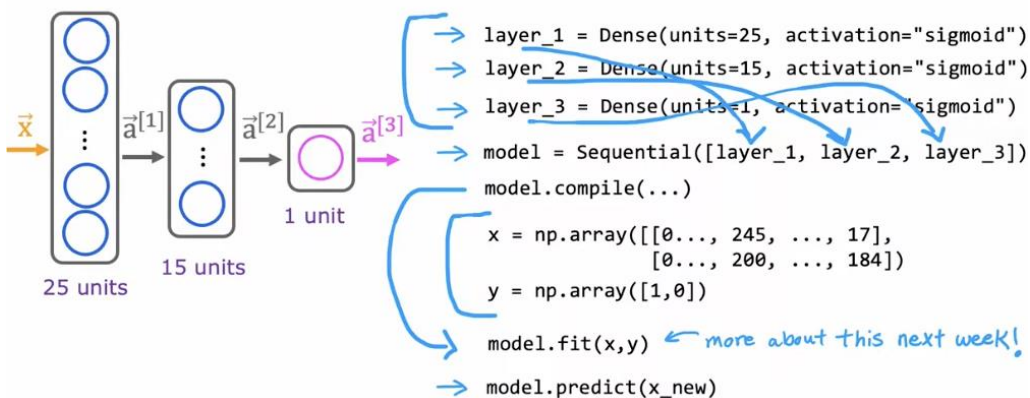
where TensorFlow adjusts the weights to reduce prediction errors.

Finally, once trained, the model can classify new digit images using:

```
model.predict(x_new).
```

The key idea is that **the image is processed layer by layer, from raw pixels to a final probability**, which is then used to make a classification decision.

# Digit classification model



## Digit Classification Model (Putting Everything Together)

This slide shows how a **digit classification neural network** is fully built and used in TensorFlow.

The **input  $x$**  represents digit images after preprocessing. Each image is converted into a **vector of pixel values**, and each row in  $x$  corresponds to one image. The target array  $y$  contains the correct labels (for example, 0 or 1) for each image.

The model is made of **three Dense layers**:

### •Layer 1 (25 units):

`Dense(units=25, activation="sigmoid")`

This hidden layer learns basic patterns from the pixel values.

### •Layer 2 (15 units):

`Dense(units=15, activation="sigmoid")`

This hidden layer combines earlier patterns into more meaningful features.

### •Layer 3 (1 unit):

`Dense(units=1, activation="sigmoid")`

This is the output layer and produces  $\vec{a}^{[3]}$ , which represents the **probability that the image belongs to the target digit**.

These layers are connected together using:

`model = Sequential([layer_1, layer_2, layer_3]).`

This means data flows **in order**, from the input layer to the output layer.

Before training, we prepare the model using:

`model.compile(...).`

This step tells TensorFlow how learning will happen (details are covered later).

The model is trained using:

`model.fit(x, y).`

During training, TensorFlow adjusts the weights so the predictions match the correct labels.

After training, the model can be used to classify new images using:

`model.predict(x_new)`.

**Key idea:** this slide connects the **neural network diagram**, the **TensorFlow code**, and the **learning process** into one complete digit classification model.

# Neural Network Implementation in Python

## Forward Prop in a single Layer

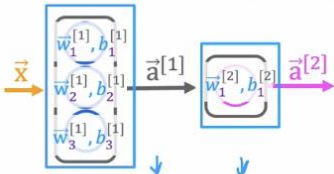
Andrew Ng

### Neural Network Implementation in Python

#### Forward Propagation in a Single Layer

This section shows how **forward propagation for one neural network layer** is implemented step by step in Python, using weights, bias, and an activation function to compute the layer's output.

# forward prop (coffee roasting model)



`x = np.array([200, 17])`

1D arrays

$$a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

`w1_1 = np.array([1, 2])`

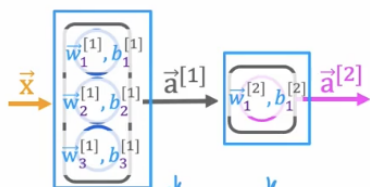
`b1_1 = np.array([-1])`

`z1_1 = np.dot(w1_1, x) + b1_1`

`a1_1 = sigmoid(z1_1)`

$w_1^{[2]}$   $w2\_1$

# forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

1D arrays

$$a_1^{[1]} = g(\bar{w}_1^{[1]} \cdot \bar{x} + b_1^{[1]})$$

$$a_2^{[1]} = g(\bar{w}_2^{[1]} \cdot \bar{x} + b_2^{[1]})$$

$w1\_1 = \text{np.array}([1, 2])$

$w1\_2 = \text{np.array}([-3, 4])$

$b1\_1 = \text{np.array}([-1])$

$b1\_2 = \text{np.array}([1])$

$z1\_1 = \text{np.dot}(w1\_1, x) + b1\_1$

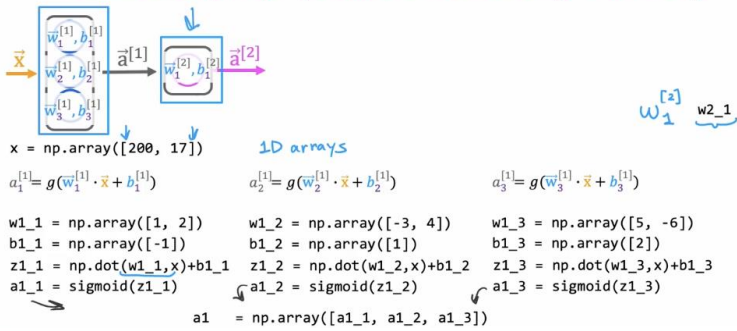
$z1\_2 = \text{np.dot}(w1\_2, x) + b1\_2$

$a1\_1 = \text{sigmoid}(z1\_1)$

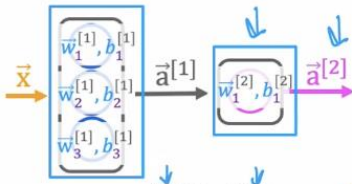
$a1\_2 = \text{sigmoid}(z1\_2)$

$w_1^{[2]}$   $w2\_1$

## forward prop (coffee roasting model)



# forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

$$a_1^{[1]} = g(\bar{w}_1^{[1]} \cdot \bar{x} + b_1^{[1]})$$

$w1\_1 = \text{np.array}([1, 2])$

$b1\_1 = \text{np.array}([-1])$

$z1\_1 = \text{np.dot}(w1\_1, x) + b1\_1$

$a1\_1 = \text{sigmoid}(z1\_1)$

1D arrays

$$a_2^{[1]} = g(\bar{w}_2^{[1]} \cdot \bar{x} + b_2^{[1]})$$

$w1\_2 = \text{np.array}([-3, 4])$

$b1\_2 = \text{np.array}([1])$

$z1\_2 = \text{np.dot}(w1\_2, x) + b1\_2$

$a1\_2 = \text{sigmoid}(z1\_2)$

$a1 = \text{np.array}([a1\_1, a1\_2, a1\_3])$

$$a_1^{[2]} = g(\bar{w}_1^{[2]} \cdot \bar{a}^{[1]} + b_1^{[2]})$$

$\Rightarrow w2\_1 = \text{np.array}([-7, 8, 9])$

$\Rightarrow b2\_1 = \text{np.array}([3])$

$\Rightarrow z2\_1 = \text{np.dot}(w2\_1, a1) + b2\_1$

$a2\_1 = \text{sigmoid}(z2\_1)$

$w1^{[2]}$   $w2\_1$